

OVERNET. presents
upgrade your digital skills

Solution Architecture

for  UniCredit

by Antonio Piemontese

October 2026



Who We Are

Training partner and instructor profile for the AI Architecture course

OVERNET

upgrade your digital skills

OverNet Education

Italian ICT training, certification and technology advisory provider.

More than 30 years of experience, with roots in Mondadori Informatica Education

Courses, corporate academies and TechAdvisory services

Official certification services, including Pearson VUE and PeopleCert

Organizer of major technology events such as [WPC](#) and Microsoft AI Tour

Headquartered in Assago (Milan)

Italy's largest ICT training company, with the broadest customer base and course portfolio.

TRAINING & ADVISORY



Antonio Piemontese

Senior IT Consultant & Course Trainer

Based in Turin, Italy

42 years of professional IT experience; freelance since 2004

Degree in Computer Science, 110/110 cum laude

Specialized in AI & ML in 2014-2019 at Politecnico di Torino

Active in Data Science and AI-related professional work and training

Professional certifications include AI, PMP, ITIL v3 Foundation and COSMIC Foundation

Has been working with Overnet since 2014

Course focus

AI & GenAI

Data Science

ML

Architecture

For any further request in training or consulting please refer to Overnet

Course Schedule

Same course hours, shown in Italian and Romanian local time

ItalyLocal time

09:00 – 12:30
Morning session

13:30 – 17:00
Afternoon session

Romania+1 hour

10:00 – 13:30
Morning session

14:30 – 18:00
Afternoon session

Two points to confirm

1

Break duration?
Proposal: 10 minutes mid-morning and 10 minutes mid-afternoon, because the programme is quite extensive.
Alternative: I am happy to make each break 15 minutes if preferred.

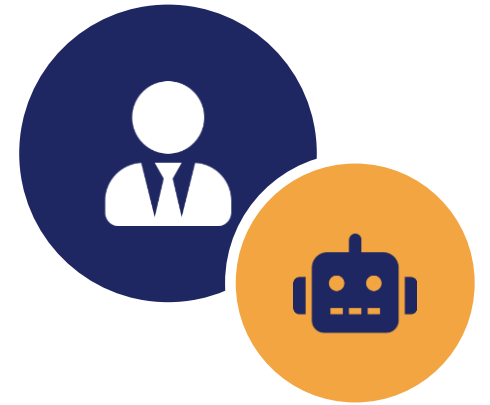
2

Lunch break timing?
Current plan: 12:30–13:30 in Italy / 13:30–14:30 in Romania.
The lunch break can be moved if another timing is more convenient.

Please let me know your preference on break duration and lunch timing.

How AI helped build this course

In preparing this course, the instructor made extensive use of AI tools, primarily **ChatGPT (GPT-5.6 Sol Pro or GPT-6 Pro)** and **Claude (Fable 5.1 Max)**. They were used as assistants, never as authors.



01

Directed

Every output came from targeted, repeated prompts written by the instructor to pursue precise teaching goals.



02

Corrected

Direction and content were continuously reviewed, corrected and refined: nothing was accepted as-is.



03

Owned

The instructor verified all materials and remains the sole and final person responsible for their content.



Why this statement

Issued in line with the transparency obligations of the EU AI Act (Regulation (EU) 2024/1689, Article 50), applicable since 2 August 2026 and confirmed on schedule by the 2026 Digital Omnibus.

AI Architecture & Solution Architecture

Two complementary architecture courses for enterprise teams

AI Architecture

Sep 28 • Oct 2

- AI and Generative AI fundamentals
- Designing AI-enabled solutions
- Generative AI architecture patterns
- Large Language Models (LLMs) and their capabilities
- Retrieval-Augmented Generation (RAG) architectures
- AI agents and multi-agent systems
- Prompt engineering and prompt management
- Vector databases and embeddings
- AI model selection and evaluation
- AI security, privacy, and responsible AI
- AI governance and compliance
- MLOps and LLMOps fundamentals
- Building AI applications on cloud platforms (GCP, Azure, AWS)
- AI integration with enterprise systems and APIs
- Monitoring, observability, and cost management for AI workloads

Solution Architecture

Oct 9 • Oct 16

- Solution Architecture fundamentals
- Architectural design patterns and best practices
- Cloud-native application architecture
- Microservices and event-driven architectures (Docker and Kubernetes)
- API (Application Program Interface) design and integration patterns
- Security by design and Zero Trust principles
- High availability, resilience, and disaster recovery
- Scalability and performance optimization
- Infrastructure as Code (Terraform)
- Architecture governance and decision frameworks
- Architecture documentation and diagrams
- Enterprise integration patterns
- Multi-cloud and hybrid-cloud architectures (vs on-premise architecture): Azure by Microsoft, AWS by Amazon, Google Cloud.

CHAPTER 01

Solution Architecture Fundamentals

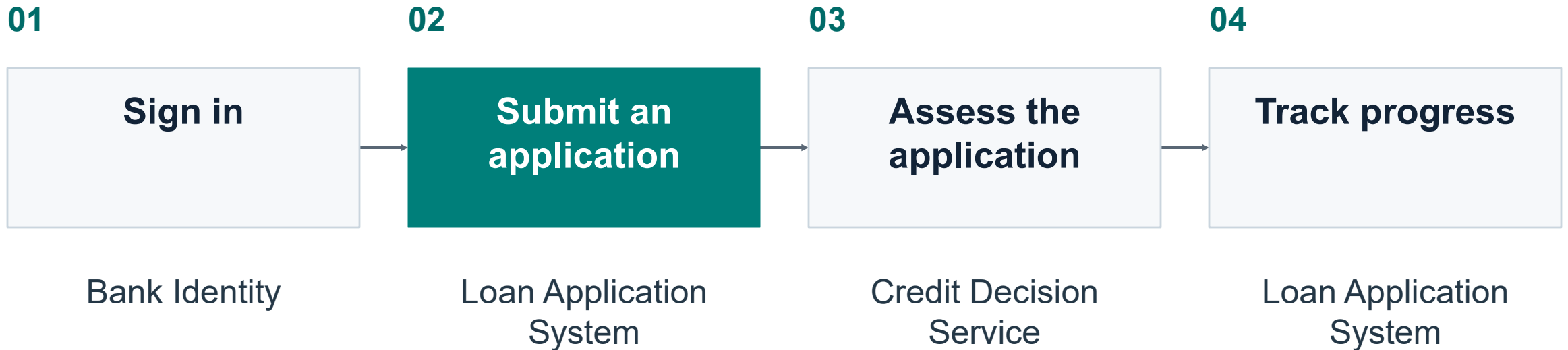
For Business and Technical Analysts
in international commercial banking

Antonio Piemontese

October 2026



A customer journey across systems



Tracking starts after receipt, while assessment continues.

Discussion

Who owns the customer experience when one dependency is slow?

Our case: digital loan applications

Business brief

Existing retail customers in one pilot country can apply for a personal loan and follow its progress.

Existing dependencies

Bank Identity handles sign-in.

Customer Records / Core supplies data.

Credit Decision Service owns credit assessment and policy.

Pilot scope

Capture, save, submit and track applications.

Architecture and its description

Solution architecture connects a business outcome to responsibilities, relationships and significant choices.

An architecture description makes those choices visible to its stakeholders.

Business outcome

What must improve, and for whom?

Responsibilities

Which parts own which behaviour and data?

Relationships

How do those parts collaborate?

Significant choices

Which decisions shape quality and change?

Architecture and detailed design

Architecture focus

Ownership of an application record

Interaction with credit assessment

Recovery after a dependency fails

Detailed design focus

Local validation implementation

Internal class or function structure (technical design and software)

Form layout and field presentation (User Interface)

The distinction depends on a decision's scope, impact and cost of change.

The distinction sometimes blurs and there can be «gray zones»

Enterprise, solution and software scope

Scope	Typical focus	Example in our case
Enterprise	Direction across the organisation	A shared-service "reuse" principle
Solution	A business outcome across systems	Loan application journey and dependencies
Software	Structure within a software system	Internal application responsibilities

These scopes can overlap. They do not prescribe a universal organisation chart.

Stakeholders and their expectations / concerns

Stakeholders	Questions the solution must address
Customers	Did the bank receive my application? What happens next? Why are not they ansering
Product and business owners	Does the journey support the intended customer outcome? Is the customer receiving the expected support?
Service owners and delivery teams	Are responsibilities, dependencies and change impacts clear?
Operations, security and risk	Can we recover, protect information and explain what happened? (the system has gone into production)

A concern states what matters. The architecture must make its response visible.

Business and technical analysts

Business analysis questions

What does an application receipt mean?

Which statuses can a customer act on?

Which outcome and rules guide the choice?

Technical analysis questions

Which system owns each responsibility?

How do interfaces behave during failure?

What evidence would validate the design?

Shared output: a decision linked to a business need and testable evidence.

Architectural drivers

A driver is a requirement or constraint that materially changes the solution's structure.

In this case, a clear receipt and durable record shape the submission flow.

Business outcome

Customers can see that an application exists.

Important function

Save, submit and track an application.

Quality requirement

Acknowledge receipt within an agreed target.

Constraint

Keep credit policy in Credit Decision Service.

Functional and non-functional requirements

Functional requirements

Save an incomplete application.

Submit valid application information.

Display the latest processing status.

Non-functional requirements (-ilities).
Frequently not made explicit / not documented / nobody take them in charge appropriately

How quickly must receipt be confirmed?

What must happen after a restart?

How much downtime is acceptable?

Complete requirements describe behaviour and the conditions it must satisfy.

Quality attribute scenarios

01 Source

Who or what causes the event?

03 Environment

Under which operating conditions?

05 Response

How must the system respond?

02 Stimulus

What happens?

04 Artifact

Which part of the system is affected?

06 Response measure

How will we evaluate the response?

Six-part scenario structure from SEI. The next slide applies it to our hypothetical pilot.

A measurable acknowledgement target

Scenario part	Digital loan application example
Source	Authenticated customer
Stimulus	A valid application submission
Environment	20 valid submissions/s, healthy intake and storage
Artifact	Loan Application System API
Response	Durably record the application, then return a reference
Response measure	p95 API acknowledgement time ≤ 2 seconds

The target measures application receipt. Credit approval happens separately.

Constraints, assumptions and decisions

Every important statement needs a type, an owner and a reason to revisit it.

Classifying statements prevents an untested assumption from becoming a hidden requirement.

Constraint

Reuse the existing Credit Decision Service.

Assumption *(often not made explicit)*

The pilot workload is 20 submissions / sec.

The API with other systems will be trivial to define

Proposed decision

Acknowledge before assessment completes.

Open question

How long may assessment remain pending?

The solution boundary

Inside the new system

Capture and save application details

Record submissions and coordinate requests

Expose the reference and processing status

Existing responsibilities

Bank Identity: customer sign-in

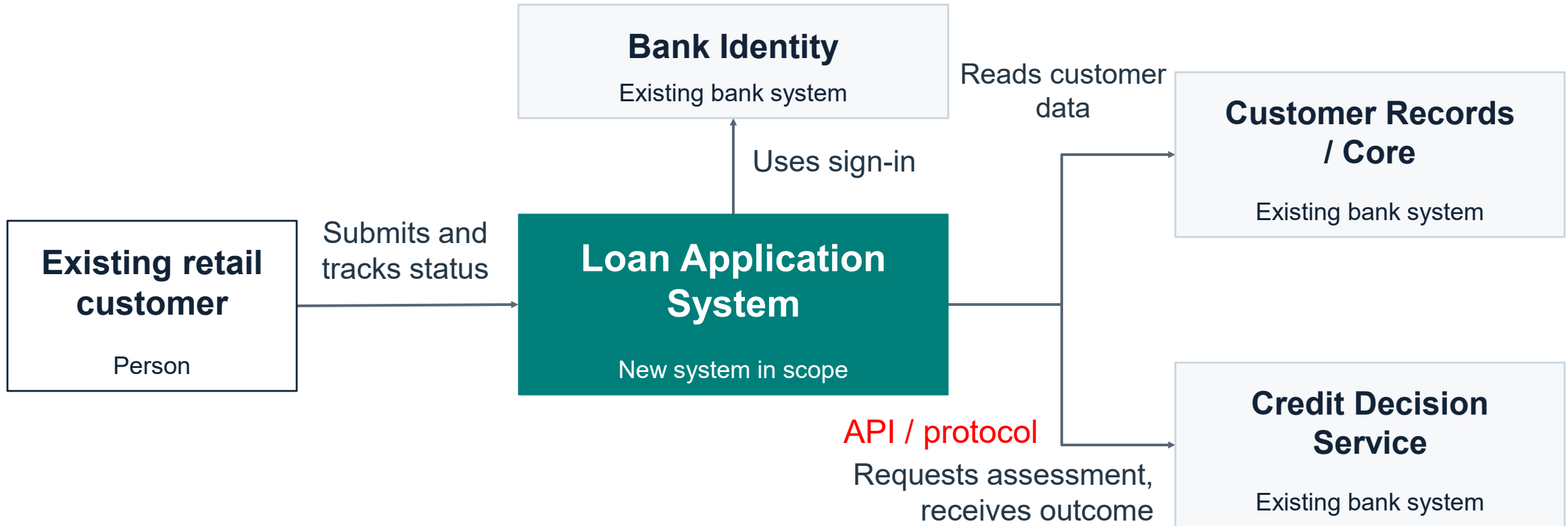
Customer Records / Core: customer data

Credit Decision Service: policy and assessment

Outside the solution boundary can still mean inside the bank.

Loan application system: context

First define the interfaces among applications. Do NOT start with analysing componets / subsystems.



Teal labels the system in scope. All dependencies shown belong to the bank in this hypothetical case.

Views for different questions

Question	Useful view	What it makes visible
Who interacts with the system?	System context	People and neighbouring systems
What happens during submission?	Interaction or dynamic	Sequence and failure behaviour
Who owns information?	Data and information	Authority and information exchanges
Where does it run and recover?	Deployment	Runtime placement and dependencies

A useful view names its scope and answers an explicit stakeholder question.

Views and viewpoints

Viewpoint

Defines concerns and conventions.

Describes how a view should be made.

Example: system context conventions.

View

Applies those conventions to a solution.

Shows selected aspects for its audience.

Example: our loan application context.

The viewpoint guides the description. The view describes this particular solution.

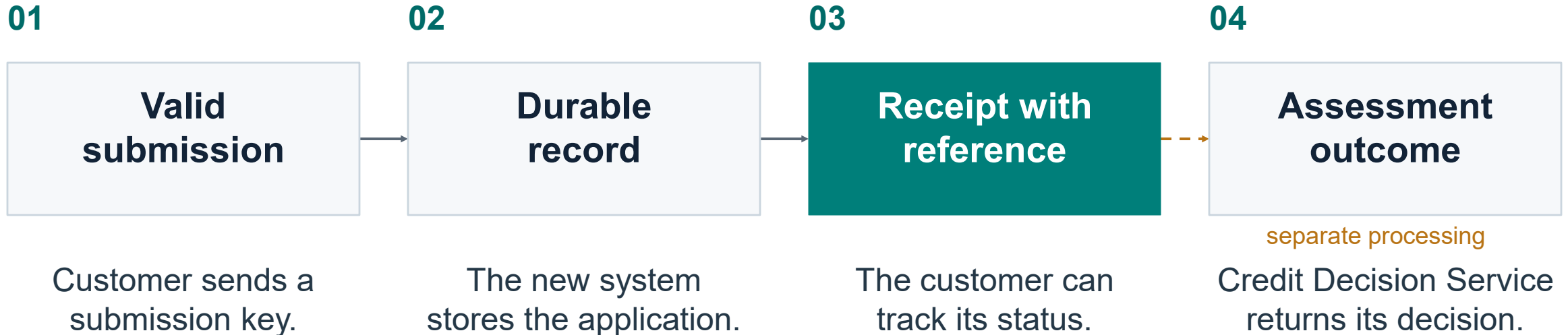
Levels of architectural detail

A **software container** is a lightweight, standalone, and executable unit – that you can **deploy** in your target machines in one step - that packages an application's code, runtime, databases, libraries, and dependencies together so it can run reliably and autonomously across different computing environments.

C4 level	What the view shows	Useful question
System context	A system and its neighbours (the internal or external apps/systems our app will interact with)	What is in scope?
Container	Applications and data stores	How is the system organised?
Component	Responsibilities within a container	How do its major parts collaborate?
Code	Implementation elements	Which detail needs explanation?

Use only useful levels. A C4 container means an application or data store.

The submission and assessment flow



During a delay

The customer sees “Pending assessment”. A timeout never implies loan approval.

Data and interface responsibilities

Information	Authoritative owner	Contract to agree
Application and status	Loan Application System	Reference, status and access rules
Customer information	Customer Records / Core	Customer ID and required attributes
Credit decision	Credit Decision Service	Outcome, correlation and version
Submission retry identity	Loan Application System	Customer, key, payload and retention



Agreement covers incoming / outgoing data, technical access, API / protocols, error behaviour and change compatibility.

Comparing feasible options

**An option is defensible
when its fit and
consequences are clear
against agreed criteria.**

Compare the same scenario and workload for
each candidate.

Feasibility

Does it satisfy the fixed constraints?

Fitness

Does it meet the important requirements?

Consequences

What complexity and operating work follow?

Evidence

Which assumptions still need validation?

Waiting versus acknowledging receipt

A Wait for assessment

Return after Credit Decision Service replies.

Customer waiting depends on assessment.

Simple visible flow, difficult long delays.

B Record, then acknowledge

Return a reference after durable recording.

Track “Pending assessment” separately.

Adds recovery and tracking of pending work.

Option B fits this pilot if storage, retry and recovery behaviour meet the requirements.

An architecture decision record

ADR-001 Application acknowledgement

Status: Proposed

Context

Customers need a prompt receipt even when assessment is delayed. The existing service keeps credit policy.

Decision

Durably record the application and return its reference. Track assessment separately as “Pending assessment”.

Consequences

Receipt does not wait for credit assessment.

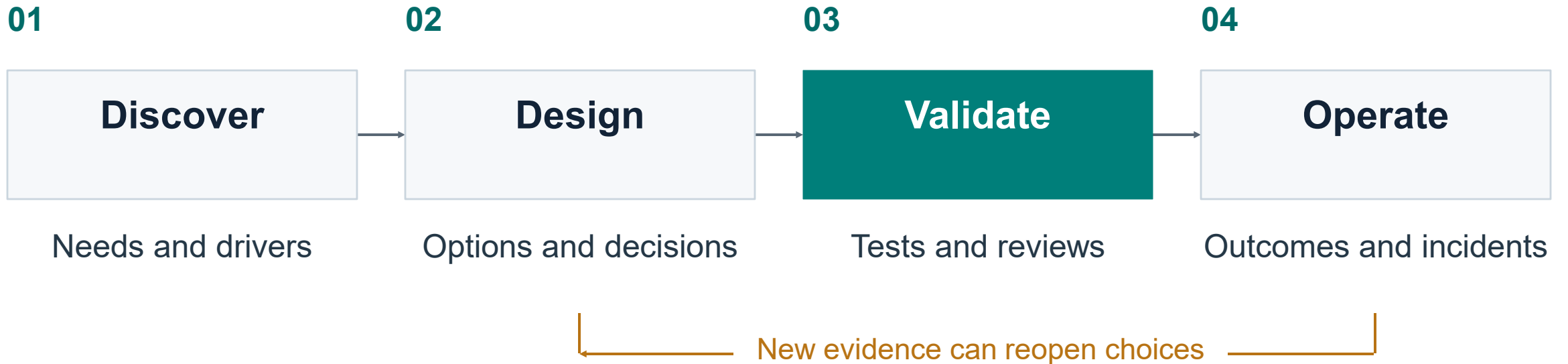
We must manage retries, recovery and status visibility.

Risks and validation evidence

Risk	Proposed response	Evidence to obtain
Duplicate application	Reuse the submission key and reference	Retry after a lost response
Lost processing after restart	Recover acknowledged application records	Restart and reconcile pending work
Delayed credit assessment	Keep “Pending assessment” visible	Simulate a five-minute service outage
Unauthorised status access	Check access to each application	Attempt access as another customer

Each risk needs an owner, a specific failure model and observable evidence.

Architecture through the lifecycle



Review points

Named owners review decisions when assumptions, evidence or business needs change.

Traceability from need to evidence

Business concern	Requirement	Decision	Evidence
Uncertain receipt	Reference after durable recording	Separate receipt from assessment	Load and restart tests
Repeated submission	Same valid retry returns the same reference	Agree submission-key semantics	Lost-response retry test
Unclear progress	Expose “Pending assessment” during delay	Store and serve processing status	Outage and status test

A change in a requirement should lead to a review of the linked decision and test.

Exercise: a delayed assessment

Scenario

Credit Decision Service is down for five minutes.

A customer repeats a valid submission after a lost response.

Intake and storage stay healthy.

Target: p95 API acknowledgement ≤ 2 s at 20 valid submissions/s.

Your group's output

- 1 State a measurable quality scenario.
- 2 Choose A or B and explain why.
- 3 Write the exact customer message.
- 4 Name one consequence and one test.

Debrief: a defensible decision

CUSTOMER MESSAGE

“Application received. Reference L-1042. Status: Pending assessment.”

Choice and condition	B: record and acknowledge while assessment is unavailable.
Retry behaviour	Within the agreed retention window, the same customer, key and payload return the same reference.
Consequence and evidence	Recover pending work. Test the outage, retry and restart paths.

The decision is conditional on evidence. Receipt confirms recording. The Credit Decision Service determines the outcome.

References and further reading

Architecture descriptions and views

ISO/IEC/IEEE 42010:2022 and Simon Brown's C4 model

- [ISO 42010 public overview](#) [C4 model diagrams](#)

Quality requirements and evaluation

Carnegie Mellon Software Engineering Institute

- [Quality attribute scenarios](#) [Quality Attribute Workshops](#)

Decisions and continuing design

Michael Nygard and the AWS Well-Architected Framework

- [Documenting architecture decisions](#) [General design principles](#)

Primary sources. Additional pattern references and teaching context appear in the speaker notes.